

1.7. Mot — un texte et sa place, sous un nom

`#include <Mot>` : Mot SALUT = { colonne, ligne, "SALUT" }; puis `texte(SALUT)` l'écrit et `effacer(SALUT)` l'efface.

1. Ce qu'il faut comprendre

Ce que c'est : un **Mot** range la **colonne**, la **ligne** et le **texte** sous un seul nom. `texte(SALUT)` l'écrit à sa place, `effacer(SALUT)` l'efface exactement — jamais une case de trop.

La ligne à écrire : `#include <Mot>`. Il faut aussi `<texte>` et `<effacer>` pour s'en servir.

Ce qu'il coûte : le texte du mot, gravé une fois, et l'appel de `texte()` — rien n'est rangé en mémoire de travail.

Essaie : déplace le Mot en changeant seulement { 6, 8, ... }.

On retrouve Mot dans 6 leçons

[1.8](#), [1.9](#), [1.10](#), [1.11](#), [1.12](#), [2.3](#)

2. Le programme, morceau par morceau

Le programme entier de la leçon, découpé comme l'œil le lit. Il fait **316 octets** de cartouche.

Ce que le compilateur y a gravé, nommé — chaque morceau, sa taille, qui l'a demandé — et ce qu'il a laissé, parce que rien ne l'appelle :

Gravé dans la cartouche :

démarrage	49 o	← la console, à l'allumage	– la mise en route de la console
main()	33 o	← démarrage	– ta fonction
VBlank	8 o	← la console, à chaque image	– compte les images, 60 fois par seconde
AttendreImage	28 o	← image()	– attend l'image suivante
AttendreVBlank	16 o	← démarrage	– attend que l'écran soit libre
AttendreAcces	12 o	← EffacerCases, EcrireTexte	– attend de pouvoir écrire une case
EffacerCases	12 o	← effacer()	– efface des cases
EcrireTexte	10 o	← texte()	– écrit un texte, lettre par lettre
CopierTuiles	49 o	← démarrage	– copie les lettres et les dessins en mémoire vidéo
EffacerCarte	14 o	← démarrage	– vide les deux cartes de l'écran
Tuiles	80 o	← CopierTuiles	– 5 lettres sur 43
"SALUT"	5 o	← texte()	– un texte gravé

Lettres gravées : ALSTU

Pas gravé (jamais appelé) : AvancerAirs, AttendreFinVBlank, AdresseCase, EcrireNombre, Diviser, Reste, Multiplier, DecalerGauche, DecalerDroite, RangerLutins, InstallerTransfert, Hasard, EffacerFond, EffacerPanneau, LireManette, DonneesTransfert, Notes

Fonctions du langage appelées :

texte()	× 1	11 o sur place, dans main()	+ EcrireTexte (10 o)
effacer()	× 1	8 o sur place, dans main()	+ EffacerCases (12 o)
image()	× 1	3 o sur place, dans main()	+ AttendreImage (28 o)
images()	× 1	2 o sur place, dans main()	

1. CE QUE LE PROGRAMME RACONTE DE LUI-MÊME

```
// ---- #include <Mot> : un texte et sa place, sous un nom ----  
// SALUT s'écrit au départ, et s'efface à l'image 120.  
  
#include <Mot>          // Mot : colonne, ligne et texte sous un seul nom  
#include <texte>  
#include <effacer>
```

2. UNE VARIABLE — SALUT

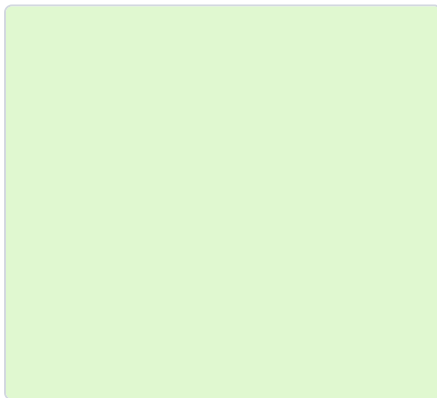
```
Mot SALUT = { 6, 8, "SALUT" }; // colonne 6, ligne 8, le texte
```

3. LE POINT D'ENTRÉE : C'EST ICI QUE LA CONSOLE ARRIVE

```
int main() {  
    texte(SALUT);           // écrit SALUT en (6, 8)  
  
    while (true) {  
        image();  
        if (images() == 120) {  
            effacer(SALUT); // efface ses 5 cases, à la bonne place  
        }  
    }  
}
```

3. Ce qui se passe, étape par étape

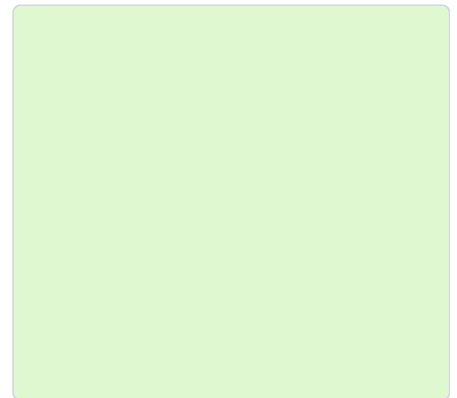
La cartouche a été compilée et exécutée pour de bon ; voici son écran aux instants qui comptent. Une image identique à la précédente n'est pas montrée — un programme qui ne bouge pas n'a qu'une seule image, et c'est la vérité sur ce programme.



la toute première image — la console vient de démarrer — image 1



un tiers de seconde plus tard : le premier dessin est posé — image 20



au bout de trois secondes — image 180

4. Ce qu'on doit voir

SALUT au milieu de l'écran, qui disparaît au bout de deux secondes.

5. Ce que la console répond

Ces contrôles ne sont pas décoratifs : ce sont ceux que `verifier-tuto.mjs` exige de cette leçon. Ils viennent d'être joués, et voici ce qu'ils ont mesuré.

- ✓ SALUT est écrit en (6, 8)
- ✓ puis effacé, à la même place

Ensuite : **1.8. Le mot et sa place sous un seul nom** — chapitre 1 : Les tout premiers pas. · gameboy3 — écrire une cartouche Game Boy en C++